SYSCRAFTERS 2026 • SYSTEM DESIGN HACKATHON

SALESTORM

High-Scale Flash Sale Architecture

Reserve without overselling. Pay without duplicate charges.
Recover without losing orders.

10,000

SIMULTANEOUS BUY REQUESTS

VS

100

AVAILABLE UNITS

Goal: valid sales never exceed stock

CORRECTNESS > AVAILABILITY

DESIGN-FIRST

Design first. Use AI to accelerate. Defend every decision.

CORRECTNESS CONTRACT

Guarantees before targets

Separate what the system must never violate from what still needs real-world load validation.

HARD GUARANTEE 01

live held + sold ≤ allocated stock

HARD GUARANTEE 02

one logical action → one business effect

FAIL-SAFE POLICY

Unknown stock authority or payment outcome → stop / reconcile, never guess

DESIGN TARGETS

~10k normal edge RPS

up to 500k peak edge sizing scenario

ASSUMPTIONS

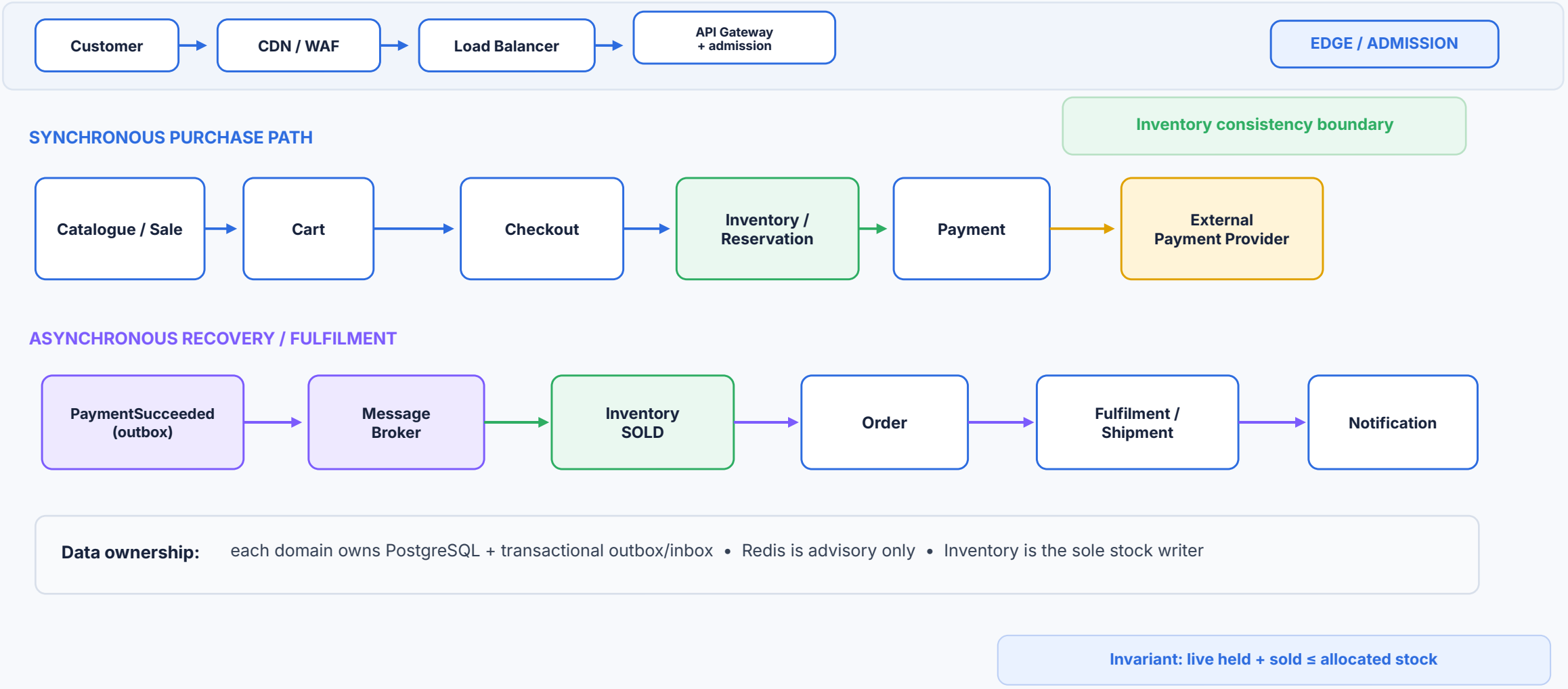
- One unit per Buy request
- One active write region for stock authority
- Payment provider supports intent lookup
- All stock writers obey the guarded protocol

Invariant: live held + sold ≤ allocated stock

SYSTEM BLUEPRINT

Architecture & ownership

The edge bounds expensive work; domain owners keep one authoritative source of truth; recovery is event-driven.



CRITICAL CONCURRENCY PATH

10,000 customers vs 100 units

Overselling is prevented at one exact point: the authoritative Inventory transaction.



THE CORRECTNESS GATE

```
UPDATE inventory
SET available = available - 1,
    reserved  = reserved  + 1
WHERE sku_id = :sku
  AND available >= 1;
```

1 row changed

RESERVE

0 rows changed → OUT_OF_STOCK

Duplicate request

Same idempotency key + same request hash → replay stored response

Expiry race

Expiry and checkout lock the same reservation. Never expire a payment-in-flight hold.

Quantity conservation

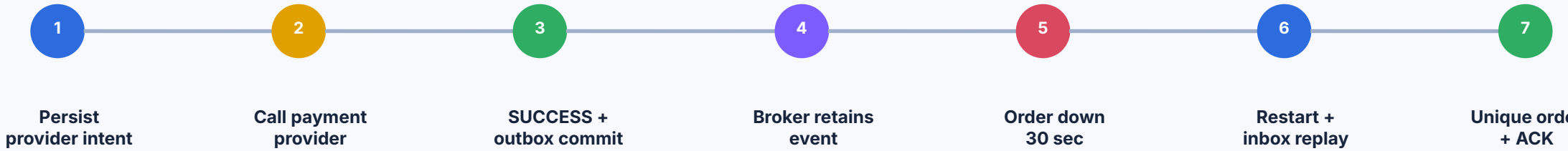
Live held + sold stays ≤ 100. Released holds may be reused; historical reservations are not the invariant.

Invariant: live held + sold ≤ allocated stock

FAILURE RECOVERY

Payment succeeds. Order is down.

A provider timeout is uncertainty — not failure. Durable facts and idempotent replay recover the workflow.



Timeout ≠ failed

Keep PAYMENT_PENDING hold. Reconcile the same provider intent before deciding.

Customer sees: PROCESSING

Do not refund just because Order is temporarily unavailable.

Replay is harmless

Inbox + unique order key make at-least-once delivery safe.

OUTBOX BROKER INBOX

Durable fact → replay → idempotent recovery

DESIGN THAT SURVIVES CHANGE

LLD, SOLID & extension points

Focused interfaces preserve failure semantics while letting providers, pricing and delivery evolve independently.

Reservation

- **ReservationService**
- InventoryRepository
- UnitOfWork
- Guarded stock operations

Payment

- **PaymentService**
- PaymentGateway
- ReconciliationWorker
- UNKNOWN-safe provider contract

Order

- **OrderService**
- OrderRepository
- StatePolicy
- Idempotent inbox consumer

HOW CHANGE ENTERS THE SYSTEM

- **Adapter** new payment / carrier provider
- **Strategy** new pricing rule
- **Repository** safe persistence abstraction
- **Outbox / Inbox** isolate side effects + replay

Interfaces may vary; safety contracts may not

SCALE, EVIDENCE & TRADE-OFF

Correctness survives the burst.

Capacity still needs live validation.

12 / 12

state-model tests passed

Duplicates • expiry races • UNKNOWN payment • redelivery • Order recovery

10,000 + retries

95 sold • 5 released • 95 orders

The simulation validates state invariants — not database throughput.

HOT-SKU LIMIT

Inventory row + durable commit

More pods do not parallelize one authoritative hot stock row.

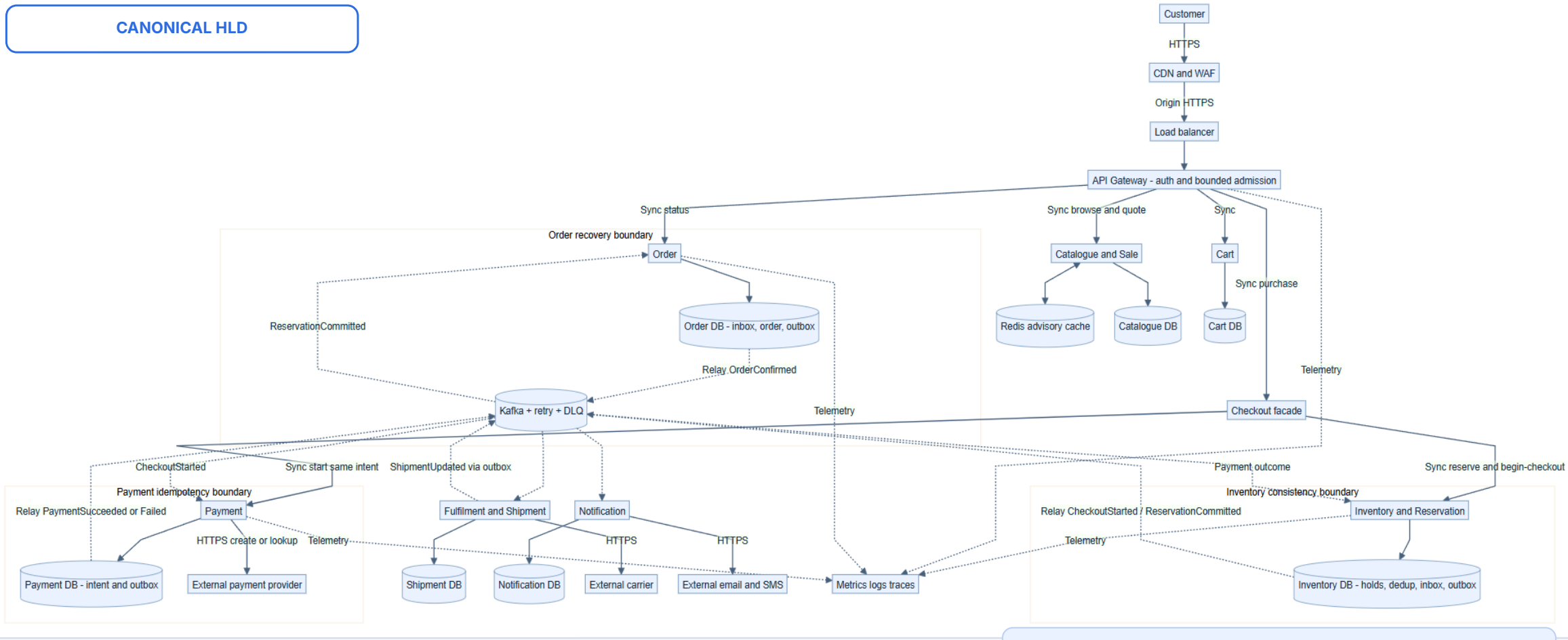
Accepted trade-off **Lower availability when stock authority or payment outcome is uncertain — because correctness is the product.**

500k edge RPS is a sizing scenario. PostgreSQL, payment-provider limits and HA behavior still require live load/failover testing.

Full high-level architecture

Canonical HLD from the Codex submission — useful for jury questions on ownership, sync/async boundaries and recovery.

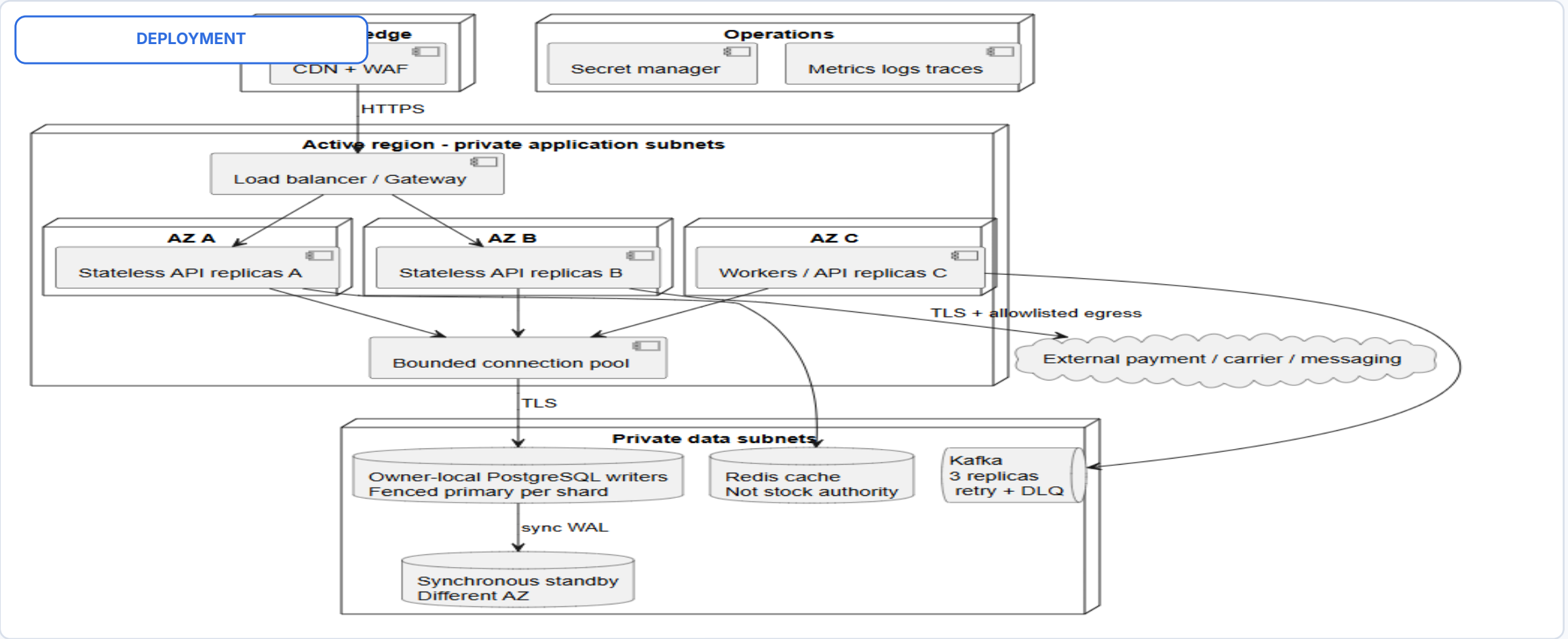
CANONICAL HLD



Invariant: live held + sold ≤ allocated stock

Deployment & scaling topology

How stateless services scale horizontally while state remains explicitly owned.

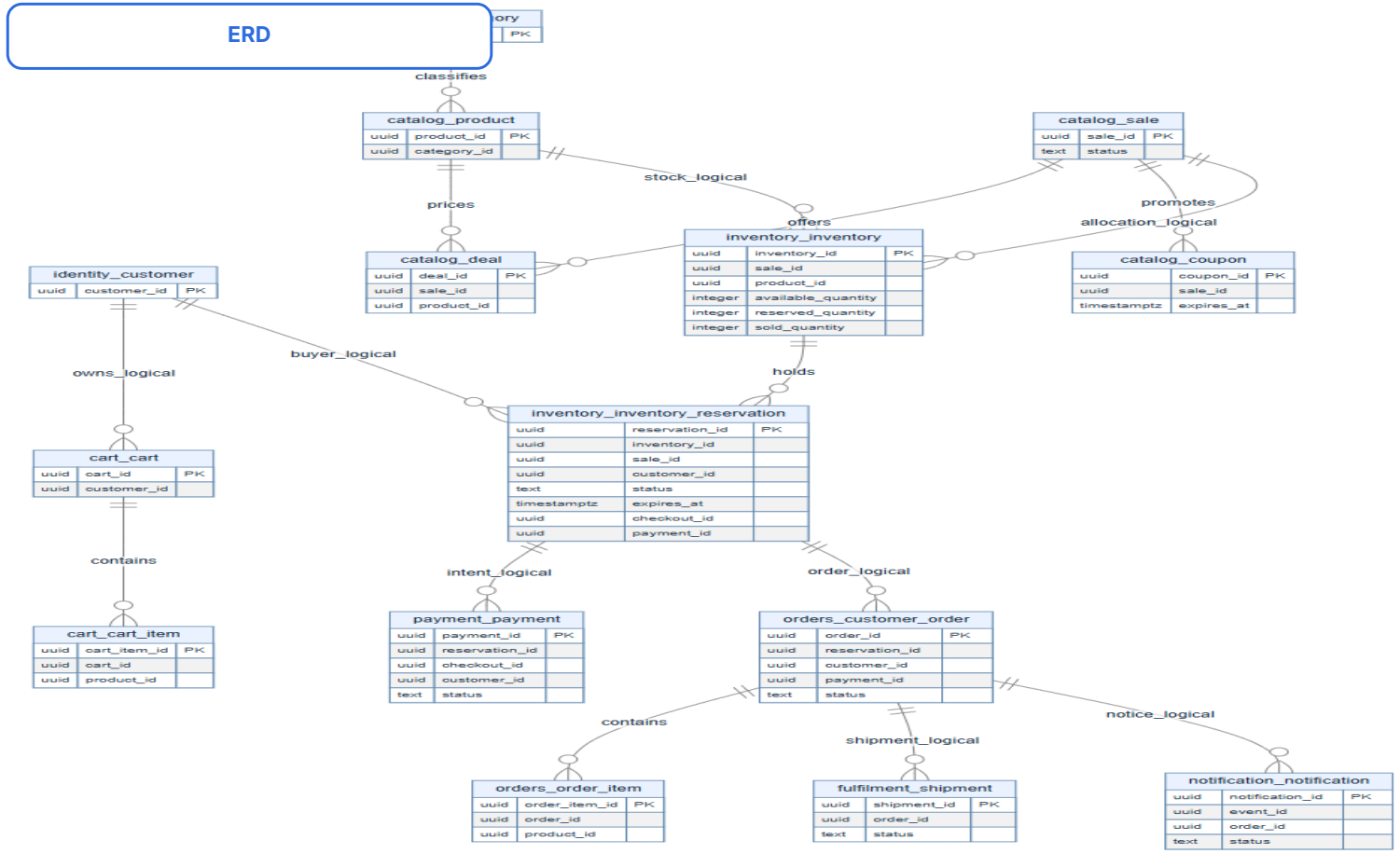


Key jury point: independent SKUs can shard; one hot SKU still serializes on the authoritative stock decision.

Database model & transaction boundary

Inventory, reservations, payments, orders and outbox/inbox records are all auditable and idempotency-aware.

ERD



Inventory transaction

Guarded stock update + reservation/hold + outbox + idempotent response commit atomically.

Uniqueness controls

Idempotency key, provider transaction reference, inbox event IDs and one-order-per-reservation constraints stop duplicate effects.

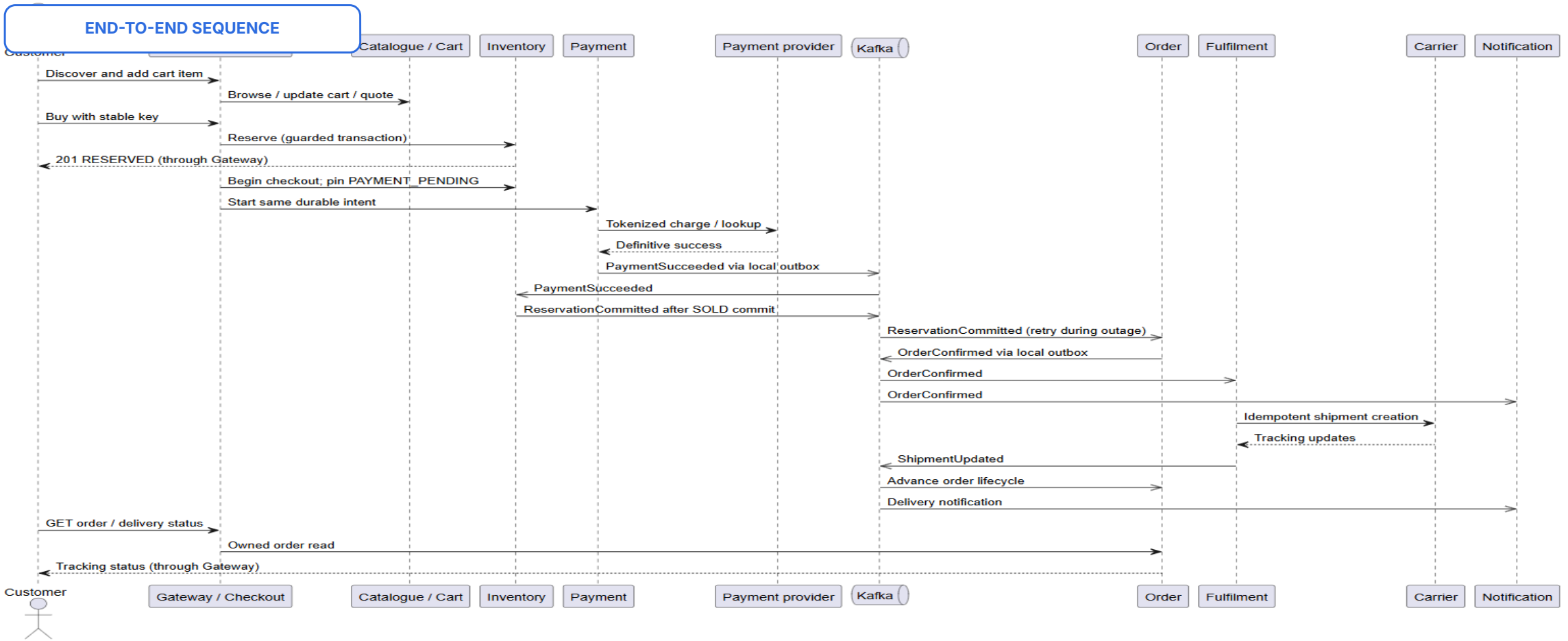
Auditability

State, timestamps, versioning and event records make distributed recovery traceable.

Complete purchase sequence

One end-to-end view from request arrival through reservation, payment, stock commitment and order creation.

END-TO-END SEQUENCE

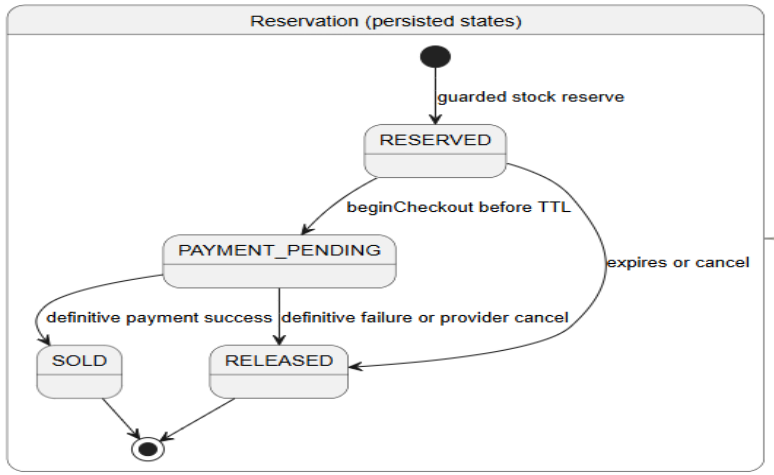


Every retry returns to an idempotent boundary

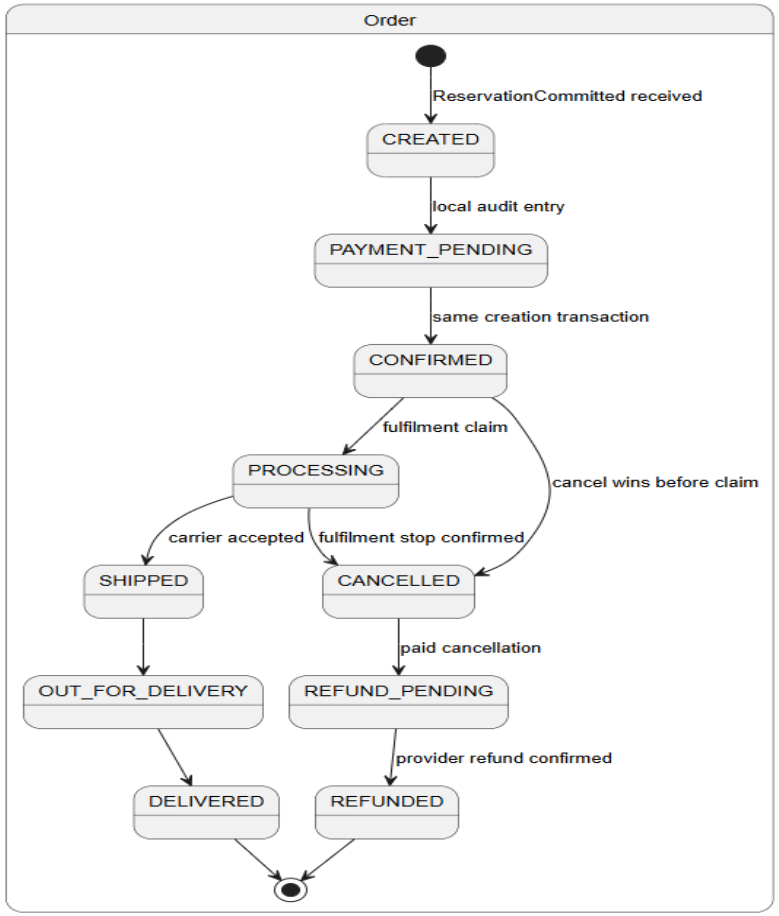
Reservation & order state model

Explicit states make timeout, expiry, compensation and replay behavior defendable.

STATE MODEL



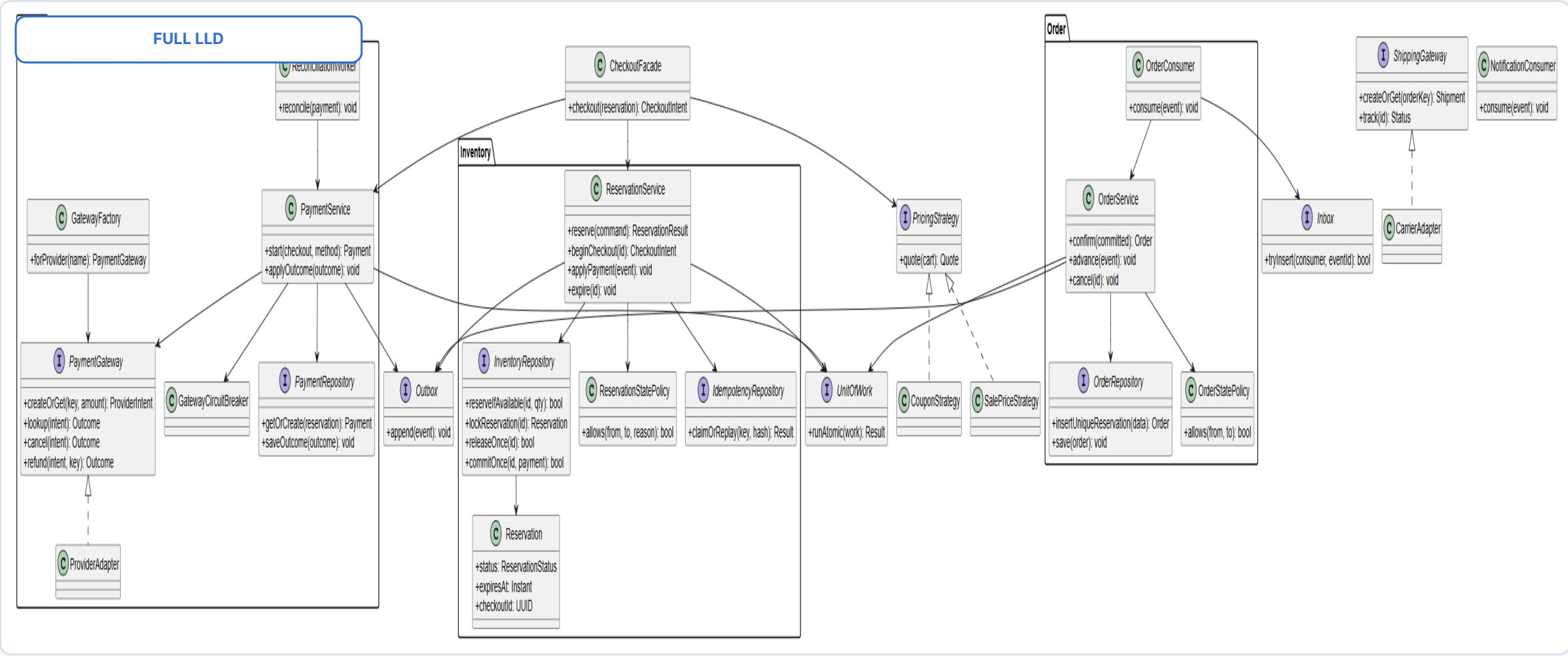
AVAILABLE is a quantity, not a reservation row.
CONFIRMED and SOLD collapse to atomic SOLD at Inventory.
PAYMENT_FAILED / TIMEOUT are release reasons, not extra states.
UNKNOWN payment retains PAYMENT_PENDING.



Jury wording: AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD, with failed/timeout paths releasing stock. CONFIRMED is the business confirmation; SOLD is durable inventory accounting in the same local flow.

Detailed LLD class diagram

Full reference view. Use the simplified Slide 6 during the 5-minute pitch.



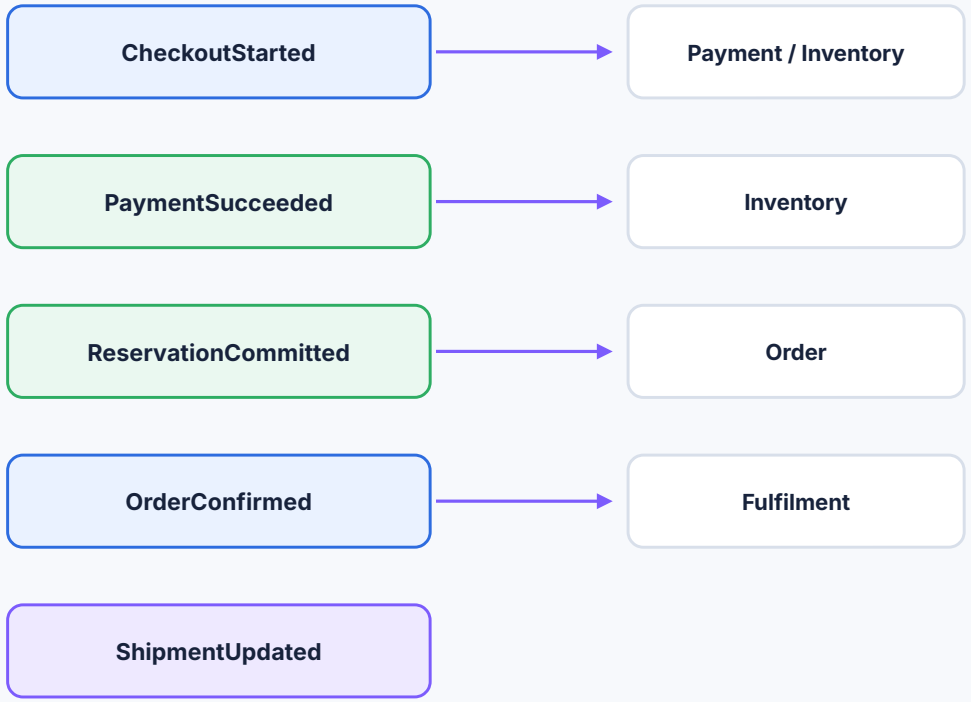
Review focus: narrow interfaces, explicit repositories, UnitOfWork, provider adapters, state policy and reconciliation worker.

API & event contract

Every externally repeatable command has explicit idempotency; every async consumer is replay-safe.

`POST /reservations`	Idempotency-Key required	**201 reserved / 409 out-of-stock**
`POST /checkout`	Starts same durable purchase intent	**Safe repeat returns original result**
`POST /payments`	One provider intent / key	**Timeout can return UNKNOWN**
`GET /orders/{id}`	Read final or processing state	**Safe polling / reconciliation**

EVENT FLOW



Delivery semantics

At-least-once delivery + inbox/event IDs + business uniqueness = safe duplicate processing.

Security, reliability & observability

The design is operable: failures are bounded, visible and recoverable.

SECURITY

- Authentication + authorization
- HTTPS / service TLS
- Rate limiting + bot / abuse protection
- Input validation + payment-data minimization
- Secrets management + audit logging

RELIABILITY

- Timeouts + bounded retries + jitter
- Circuit breaker for unstable providers
- Transactional outbox + idempotent inbox
- Dead-letter + reconciliation paths
- Graceful degradation; no unsafe fail-open

OBSERVABILITY

- Request rate / latency / error rate
- Reservation contention + stock anomalies
- Payment failure / timeout / reconciliation
- Queue lag + DLQ depth
- Trace ID: Gateway → Inventory → Payment → Order

Architecture decisions & jury-ready defense

Eight ADRs keep the design explicit — including what we chose not to optimize for.

ADR-001	**Inventory concurrency**	Guarded PostgreSQL transaction
ADR-002	**Inventory authority**	SQL source of truth; cache advisory
ADR-003	**Caching**	Cache-aside for reads; never authority
ADR-004	**Messaging**	Outbox + broker + inbox
ADR-005	**Payment workflow**	One intent + UNKNOWN reconciliation
ADR-006	**Consistency**	Strong local consistency; eventual downstream
ADR-007	**Order recovery**	Replay durable ReservationCommitted
ADR-008	**Idempotency**	Stable keys + hashes + uniqueness

If the jury asks “Why not Redis locks / queue everything / just autoscale?” **Because the design optimizes the single thing the business cannot repair later: selling more stock than exists. Scale is bounded around that authority, not substituted for it.**

Invariant: live held + sold ≤ allocated stock